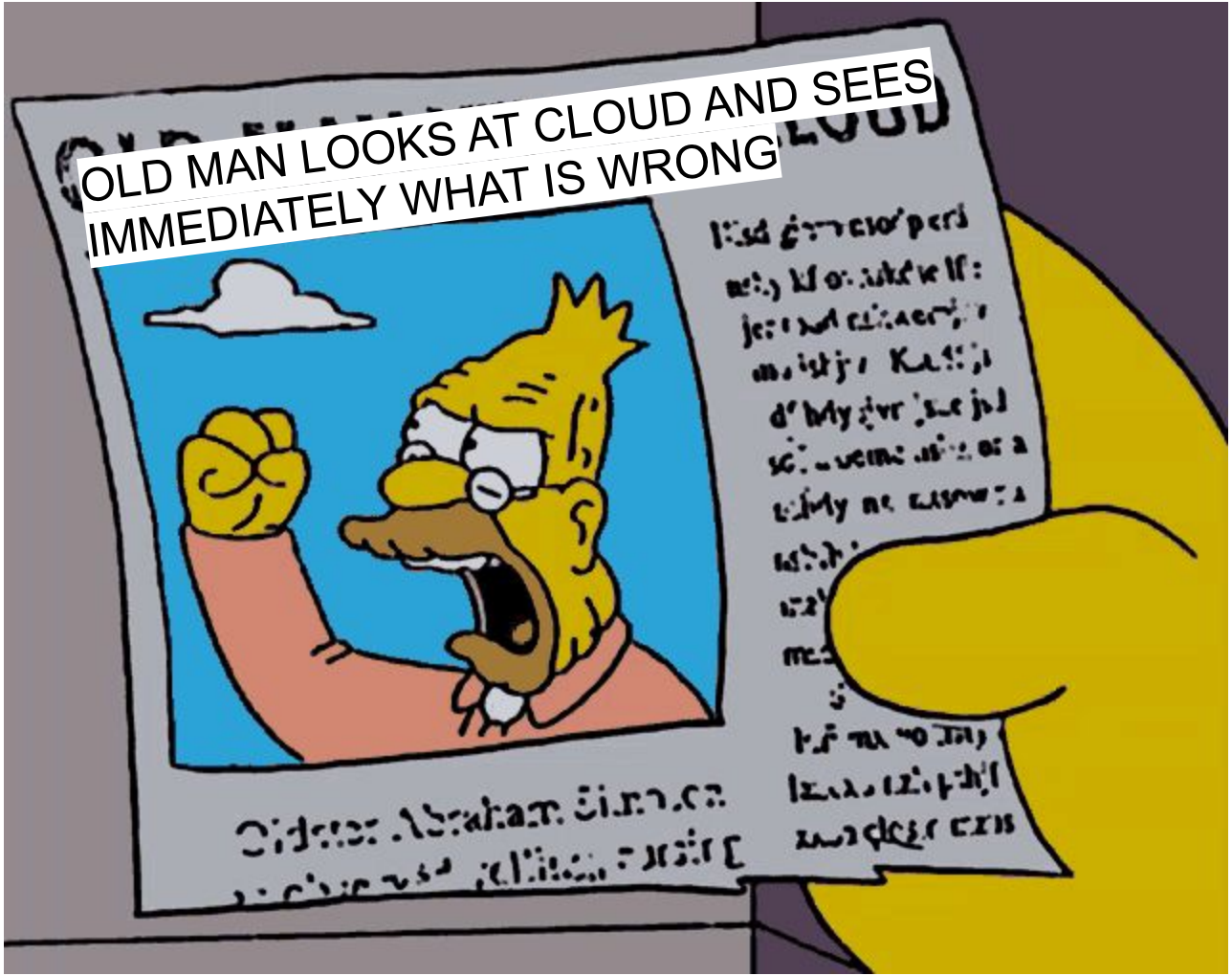
Debugging apps with Deno and OpenTelemetry

Luca Casonato

JS World Amsterdam 2025

Who here has struggled debugging their server in production?

OLD MAN YELLS AT CLOUD


OLD MAN LOOKS AT CLOUD AND SEES
IMMEDIATELY WHAT IS WRONG

But first…

I'm **Luca Casonato**

But first…

I'm **Luca Casonato**

Software engineer at **Deno**

But first…

I'm **Luca Casonato**

Software engineer at **Deno**

TC39 delegate

Co-chair of **WinterTC**

Contribute to web specs

Ujjwal from this morning

Dan, speaking in an hour

Me

A good debugging experience
requires good **data**.

What happened,
who triggered it,
how long did it take,
when did it start,
and **which** request is it a part of?

Most applications create

✨ ***Logs*** ✨

```
/tmp ❯❯❯ deno run main.ts
[http] GET /api/book/1
[http] GET /api/book/2
[db] SELECT * FROM books WHERE id = 1
[db] SELECT * FROM books WHERE id = 2
error: Uncaught (in promise) PostgresError: 'name' column is null
  throw new PostgresError(`'name' column is null`);
        ^
    at query (file:///tmp/main.ts:29:9)
    at queryBook (file:///tmp/main.ts:25:3)
    at handleRequest (file:///tmp/main.ts:21:3)
    at file:///tmp/main.ts:17:1
```

```
/tmp ❯❯❯ deno run main.ts
[http] GET /api/book/1
[http] GET /api/book/2
[db] SELECT * FROM books WHERE id = 1
[db] SELECT * FROM books WHERE id = 2
error: Uncaught (in promise) PostgresError: 'name' column is null
  throw new PostgresError(`'name' column is null`);
        ^
    at query (file:///tmp/main.ts:29:9)
    at queryBook (file:///tmp/main.ts:25:3)
    at handleRequest (file:///tmp/main.ts:21:3)
    at file:///tmp/main.ts:17:1
```

Does book 1 or book 2 have a *null* name field?

Most applications **only** create

😔 ***Logs*** 😔

Let's spice up our logs

```
/tmp ❯❯❯ deno run main.ts
[req:1] [http] GET /api/book/1
[req:2] [http] GET /api/book/2
[req:1] [db] SELECT * FROM books WHERE id = 1
[req:2] [db] SELECT * FROM books WHERE id = 2
[req:1] error: Uncaught (in promise) PostgresError: 'name' column is null
  throw new PostgresError(`'name' column is null`);
        ^
    at query (file:///tmp/main.ts:29:9)
    at queryBook (file:///tmp/main.ts:25:3)
    at handleRequest (file:///tmp/main.ts:21:3)
    at file:///tmp/main.ts:18:1
```

It was book 1!

You could pass the request ID to every function

```
1   export async function handle(req: Request): Promise<Response> {
2     const requestId : number = Math.random();
3
4     console.log( ... data: `[req:${requestId}] [http] ${req.method} ${req.url}`);
5
6     const url = new URL(req.url);
7     const bookId : string = url.pathname.split(separator: "/")[2];
8     const data : Book = await getData(requestId, bookId);
9
10    return Response.json(data);
11  }
12
13  async function getData(requestId: number, bookId: string): Promise<Book> {
14    console.log( ... data: `[req:${requestId}] [db] get book ${bookId}`);
15    try {
16      return await db.selectFrom("books").where("id", "=", bookId).first();
17    } catch (err) {
18      console.error( ... data: `[req:${requestId}] ${err}`);
19      throw err;
20    }
21  }
```

This is going to get repetitive…

Demo

This is not magic ✨.

It's called *async context propagation*.

Under the hood:

`AsyncContext`

https://github.com/tc39/proposal-async-context

We can also use this to track something else: **who** triggered something.

Why did this database query happen?

Which caller passed the wrong arguments to this function?

For this, we need to know the **parent operation**

This gets us

✨ ***Traces*** ✨

Traces

Who triggered **what**, **when,** and **how long did it take**?

Trace

book-api: GET /books/:id 235.05ms

2025-02-28 01:30:47.392 GET 200 /books/:id

Give feedback | Trace ID | Export

Span Filters

7 spans Prev Next

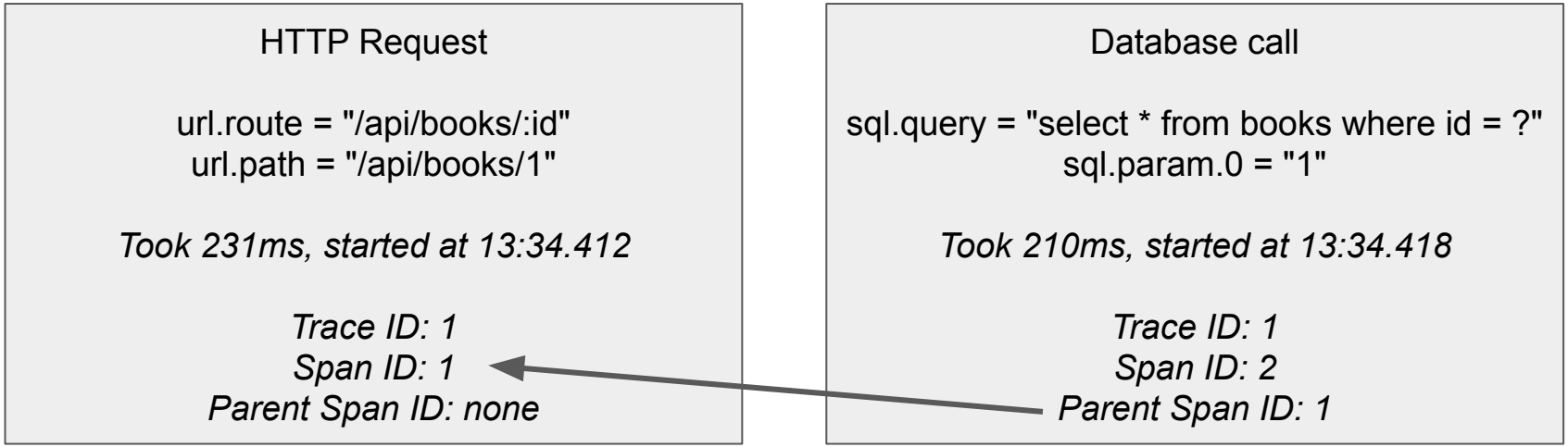
0µs 58.76ms 117.53ms 176.29ms 235.05ms

Service & Operation	0µs	58.76ms	117.53ms	176.29ms	235.05ms
book-api GET /books/:id (235.05ms)					
authorization check (10.34ms)	10.34ms				
get tokens/:token (10.2ms)	10.2ms				
get books/:id (8.18ms)	8.18ms				
SELECT * FROM books WHERE id = ? (74.57ms)	74.57ms				
SELECT * FROM authors WHERE id = ANY(?) (125.22ms)	125.22ms				
set books/:id (12.34ms)	12.34ms				

Traces are made of **spans**

Span? **One operation** which records:

- **What** the operation is (name and structured attributes)
- **How long** it took
- **When** it started
- **Who** triggered it (the parent span)
- **Which** trace it is part of


HTTP Request
url.route = "/api/books/:id"
url.path = "/api/books/1"
Took 231ms, started at 13:34.412
Trace ID: 1
Span ID: 1
Parent Span ID: none
Database call
sql.query = "select * from books where id = ?"
sql.param.0 = "1"
Took 210ms, started at 13:34.418
Trace ID: 1
Span ID: 2
Parent Span ID: 1

Having this data helps answer **why** questions about **specific requests**:

Why did this request take so long?
Why did this request error?
Why is the wrong thing happening?

Demo

Microservices anyone?

Traces can span **multiple processes**.

Reminder: spans need to know their parent

HTTP Request

url.route = "/api/books/:id"
url.path = "/api/books/1"

Took 231ms, started at 13:34.412

Trace ID: 1
Span ID: 1
Parent Span ID: none

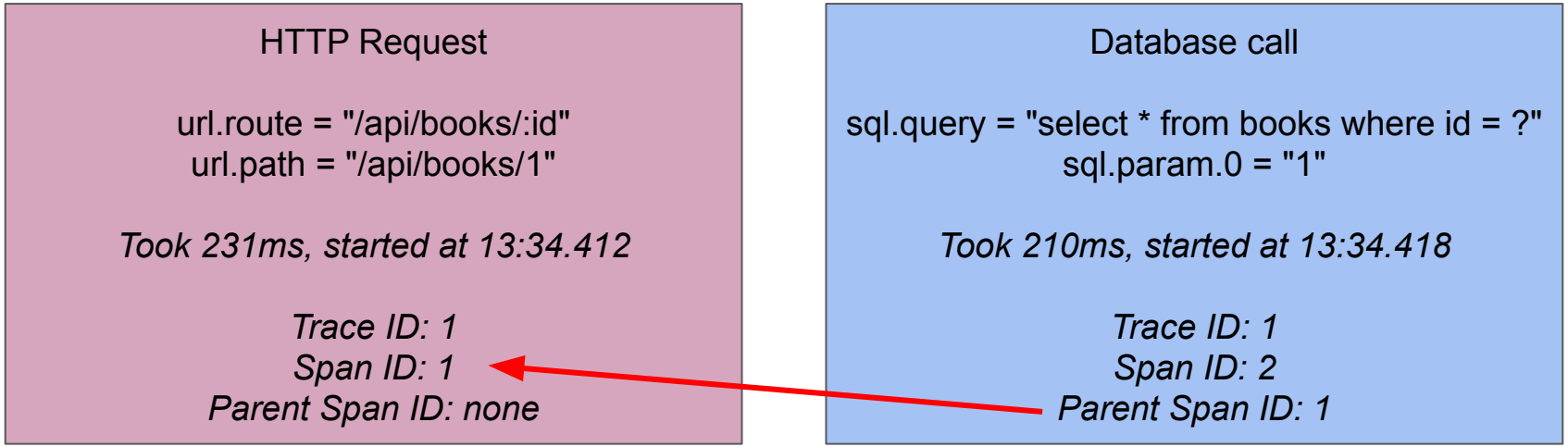
Database call

sql.query = "select * from books where id = ?"
sql.param.0 = "1"

Took 210ms, started at 13:34.418

Trace ID: 1
Span ID: 2
Parent Span ID: 1

Reminder: spans need to know their parent



Propagate span and trace ID between processes

Trace Context

W3C Recommendation 23 November 2021

§ 3.1 Relationship Between the Headers

The `traceparent` request header represents the incoming request in a tracing system in a common format, understood by all vendors. Here's an example of a `traceparent` header.

```
traceparent: 00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01
```

The `tracestate` request header includes the parent in a potentially vendor-specific format:

```
tracestate: congo=t61rcWkgMzE
```

For example, say a client and server in a system use different tracing vendors: Congo and Rojo. A client traced in the Congo system adds the following headers to an outbound HTTP request.

```
traceparent: 00-0af7651916cd43dd8448eb211c80319c-b7ad6b7169203331-01
```

The DIY approach

```
1   import { trace } from "npm:@opentelemetry/api";
2
3   export function fetchWithTrace(req: Request): Promise<Response> {
4     const span = trace.getActiveSpan();
5     if (!span) return fetch(req);
6     const spanContext = span.spanContext();
7     const traceparent = `00-${spanContext.traceId}-${spanContext.spanId}-01`;
8     req.headers.set("traceparent", traceparent);
9     return fetch(req);
10  }
11
12  // every call to `fetch` now needs to be replaced with `fetchWithTrace`
13  const resp = await fetchWithTrace(new Request("https://example.com"));
14
15
```

This is going to get repetitive… **again…**

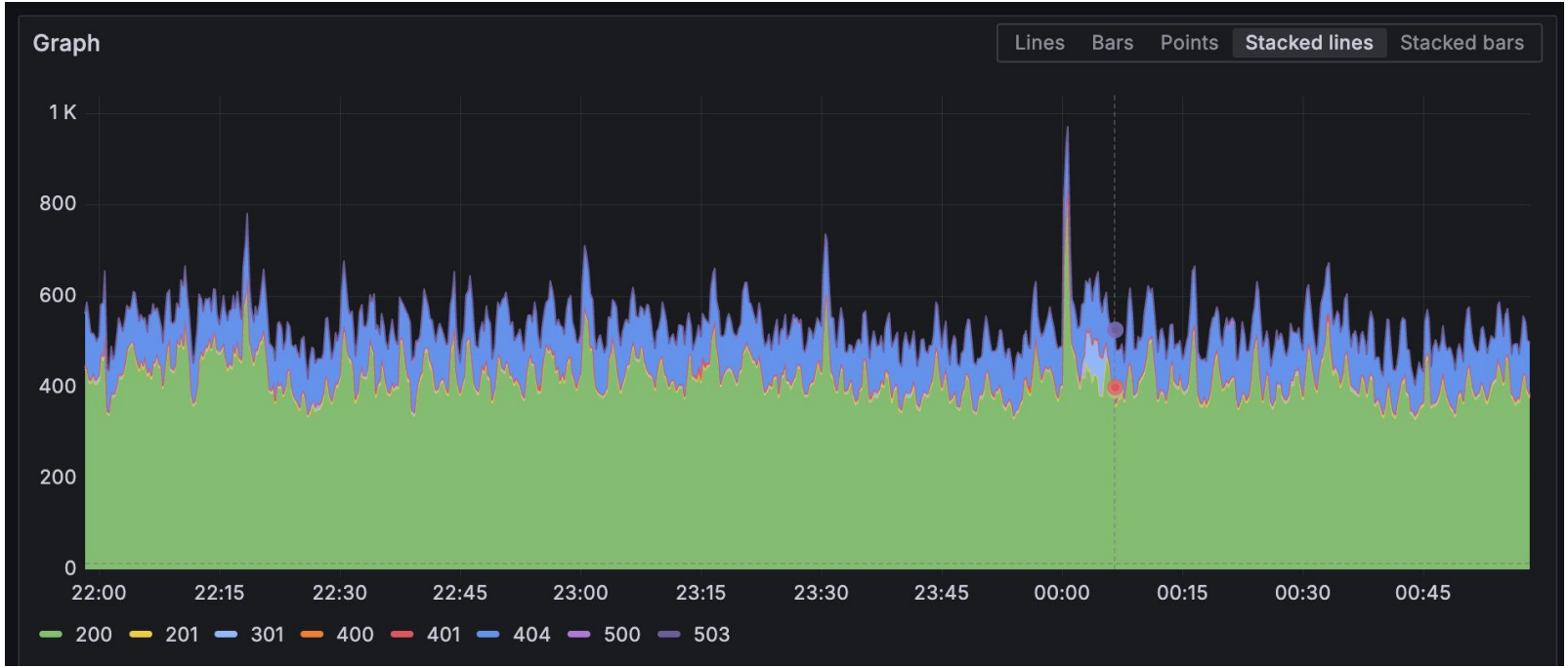
Demo

We can now debug things if they go wrong.

But how do you know if you even have a problem that needs to be investigated?

Metrics: aggregate data

How frequently is something happening?



Demo

The three pillars of **OpenTelemetry**

Logs

What happened, **when**, and *maybe* **which** request is it a part of?

Traces

What happened, **who** triggered it, **when** did it happen, **how long** did it take, and **which** request is it a part of?

Metrics

How frequently did something happen?

In Deno 2.2, **telemetry is built right in**.

Run a new or existing app in Deno and get telemetry automatically, with no setup.

Out of the box

- Log correlation with trace IDs
- One trace per HTTP request
- Spans for incoming and outgoing HTTP requests
- Request count metrics (by status code)
- Response latency (by status code)
- Automatic propagation

And more coming soon

Need more? Add your own logs, spans, and metrics.

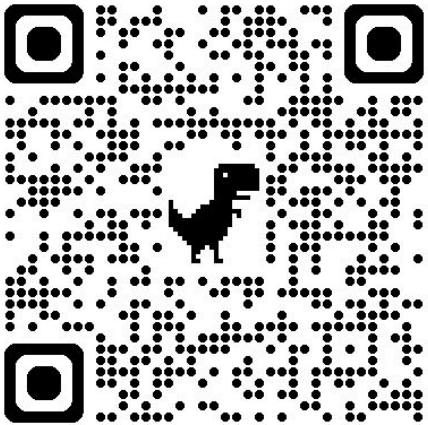
Try it out today

1. Install Deno
2. Set up Grafana (https://deno.co/otel)
3. `OTEL_DENO=true deno run --unstable-otel main.ts`
4. Go to http://localhost:3000

Or export to your existing observability stack:

- Datadog
- Honeycomb
- New Relic
- Google Cloud Monitoring

and everyone else that supports OTLP



Uncomplicate JavaScript

Deno is the open-source JavaScript runtime for the modern web.

Documentation >

GitHub >